

Evaluating the Multi-Agent Coordination Tax: Dynamic Routing Across Architectural Topologies

1st Yian Chen

Department of Electrical and Computer Engineering
Stevens Institute of Technology
Hoboken, USA
ychen17@stevens.edu

2nd Hong Man

Department of Electrical and Computer Engineering
Stevens Institute of Technology
Hoboken, USA
hman@stevens.edu

Abstract—As agentic systems scale, the prevailing industry assumption that “more agents are inherently better” becomes computationally and economically flawed. Building upon empirical scaling laws, this paper introduces a Dynamic Architecture Router designed to predictively dispatch tasks to either a Single-Agent System (SAS), Centralized Multi-Agent System (CMAS), or a Decentralized Multi-Agent System (DMAS). To evaluate this, we introduce “Routing Regret” — a composite metric of trajectory accuracy, inference latency, and token efficiency. We evaluate the router using a multi-domain offline testing framework utilizing the WorkBench, BrowseComp-Plus, and Fin-RATE datasets. Our updated LLM-backed fixed-corpus pilots reveal a continued but less extreme class imbalance in the 110-task pilot suite (79 tasks favoring SAS, 4 favoring CMAS, and 27 favoring DMAS under the composite oracle). Furthermore, our MAST-inspired semantic telemetry reveals that Multi-Agent Systems predominantly fail via “Requirement Misses” due to context dilution across graph edges. We conclude that while MAS holds measurable promise for financial comparison and longitudinal synthesis, zero-shot heuristic routing targeting single-agent topologies currently provides the most robust dispatching policy under our controlled evidence-access setting.¹

Index Terms—Multi-Agent Systems, Large Language Models, Orchestration, Dynamic Routing, Agentic Workflows, Coordination Tax

I. INTRODUCTION

A. Background and Importance

The transition from single-turn, reactive large language models (LLMs) to autonomous, multi-step agentic systems represents a fundamental paradigm shift in artificial intelligence. LLM-based agentic systems are capable of reasoning, planning, and external environment interaction through dynamic tool use, and are in turn increasingly positioned as the primary mechanism for automating complex workflows.

As enterprise adoption of generative AI accelerates, inference cost and execution latency have become paramount concerns [1]. The economic and operational viability of deploying agentic systems in production depends on compute-optimal test-time scaling [2]. If an automated workflow consumes excessive API tokens or requires too much wall-clock time to execute a simple sequential task, its utility is severely

diminished. Consequently, achieving architectural efficiency in agentic orchestration becomes critical for scalable enterprise deployment.

B. Challenges in Agentic Orchestration

Orchestrating these systems presents significant technical challenges. The prevailing design heuristic often relies on trial-and-error or the assumption that “more agents are always better,” leading developers to construct multi-agent swarms under the belief that collective reasoning is superior to individual capability [3]. However, such scaling hypothesis proves to be both computationally and economically unsustainable.

Recent technical literature has also challenged this assumption. Researchers at Google DeepMind and MIT conducted an exhaustive evaluation of 180 agent configurations, revealing that while multi-agent coordination can improve performance on highly parallelizable tasks, it significantly degrades performance on tasks that require strict sequential constraint satisfaction, with performance drops ranging from 39% to 70% [4]. Li [5] corroborates this, emphasizing that capable Single-Agent Systems (SAS) often outperform complex MAS when the administrative overhead of inter-agent communication outweighs the cognitive benefits.

A primary challenge lies in what is known as the “Multi-Agent Coordination Tax”. In a Centralized MAS, repeated state synchronization can duplicate context across supervisor-worker exchanges, producing sharp token growth and, in long runs, possible context window overflows. Recent research indicates that decentralized and hybrid MAS methods frequently suffer from fragmented context and semantic degradation, as the fragmented global state space across multiple agents can lead to ‘Requirement Misses’ and a failure to meet strict sequential constraints [5] [4]. Furthermore, failure diagnostics in multi-agent orchestration remain rudimentary, making it difficult to differentiate between a generic LLM hallucination, structural architectural problem, or a system design issue.

C. Related Work

This architectural shift and the need for intelligent orchestration are reflected across the current engineering landscape. Recent surveys emphasize the critical need to minimize latency

¹Code available at <https://github.com/yianan261/Dynamic-Architecture-Router-for-Agentic-Systems>

[6] and formalize agentic software engineering [7]. Multi-agent systems are increasingly deployed in specialized industrial applications, ranging from financial frameworks [8] and electronic design automation [9], to digital twin simulations [10], self-organizing networks [11], computer-automated design [12], and human-machine robotics [13].

To address the lack of diagnostic visibility in these deployments, researchers at UC Berkeley recently introduced the Multi-Agent System Failure Taxonomy (MAST), identifying distinct breakdown categories such as Inter-Agent Misalignment and System Design Exhaustion [14].

D. Contributions

To bridge the gap between theoretical agentic scaling laws and practical deployment, this paper introduces a Dynamic Architecture Router. Rather than relying on a static system that may fail under different task structures, our approach serves as an empirical test for orchestration theories by dispatching tasks to the most suitable architecture. Our primary contributions are:

- 1) **Dynamic Routing Engine:** We developed a predictive routing framework that classifies incoming queries based on structural complexity and dispatches them to either SAS, CMAS, or DMAS, balancing risk versus reward.
- 2) **The Routing Regret Metric:** We establish a mathematically rigorous “Composite Reward” metric to declare an Oracle architecture for any given task, measuring the exact accuracy, latency, and token regret incurred by suboptimal routing.
- 3) **Multi-Domain Offline Evaluation:** We implemented a standardized benchmarking harness adapting WorkBench [15], BrowseComp-Plus [16], and Fin-RATE [17] into offline schemas to isolate architectural capabilities from web variance.
- 4) **MAST-Inspired Failure Taxonomy:** We implemented a semantic failure taxonomy to quantitatively diagnose the causes of MAS structural breakdowns. This explains the “why” of a failure inside multi-agent systems instead of a simple binary pass/fail benchmarking.

II. PROBLEM STATEMENT

The Coordination Tax in multi-agent systems manifests in three primary stages. First, passing state between nodes requires duplicating context windows, leading to catastrophic token bloat. Second, the network overhead of invoking multiple LLMs sequentially drastically increases end-to-end inference latency. Finally, passing complex constraints through multiple intermediate agents frequently results in semantic degradation (we term this “synthesis drift”). An instruction provided by the user may be perfectly understood by the Supervisor agent, but its nuances are often lost by the time a Worker agent executes the final tool call.

In this project we design and implement three distinct agent architectures, namely Single-Agent System, Centralized Multi-Agent System, and Decentralized Multi-Agent System. See Figure 1 for the design overview. In order to quantify the

Coordination Tax, we must formally define the structural and mathematical differences between Single-Agent and Multi-Agent topologies. Let a user task T be solved using a global set of available enterprise tools O_{global} .

A. Agent System Formalism

Inspired by the agent-system formalism introduced in [4]², and to formally analyze orchestration overhead and routing regret, we define an agentic system as a tuple:

$$S = (A, E, C, \Omega) \quad (1)$$

where:

- $A = \{a_1, a_2, \dots, a_n\}$ is the set of agents,
- E is the shared execution environment,
- C defines the communication topology between agents,
- Ω represents the orchestration policy governing coordination and aggregation.

When $|A| = 1$, the system reduces to a Single-Agent System (SAS). When $|A| > 1$, the system becomes a Multi-Agent System (MAS).

Each individual agent a_i is modeled as:

$$a_i = (\Phi_i, O_i, M_i, \pi_i) \quad (2)$$

where:

- Φ_i is the reasoning policy (typically an LLM),
- O_i is the available toolset,
- M_i is the internal memory/state,
- π_i is the action-selection policy.

At timestep t , the agent receives an observation history $h_{i,t}$ and selects an action:

$$a_{i,t} = \pi_i(h_{i,t}) \quad (3)$$

The environment then returns an observation:

$$o_{i,t} = E(a_{i,t}) \quad (4)$$

and the history updates as:

$$h_{i,t+1} = f(h_{i,t}, a_{i,t}, o_{i,t}) \quad (5)$$

This iterative perception-action loop forms the central execution mechanism for both SAS and MAS architectures.

B. Formalizing Single-Agent Systems (SAS)

A Single-Agent System is defined as a unified policy where a single LLM maintains a continuous, global state space S . The agent has unrestricted access to the global toolset O_{global} . At any discrete reasoning step t , the state transition and action generation are functions of the entire historical trajectory. The action a_t is sampled from the policy π_{SAS} :

$$a_t \sim \pi_{SAS}(a_t | s_{1:t}, O_{global}) \quad (6)$$

Since the state S is monolithic, the SAS gets the full internal context retention. However, this risks context-window saturation and “tool confusion” if O_{global} is excessively large, as the model’s attention mechanism must weigh all possible tools simultaneously.

²The mathematical formulation and definitions in this section closely follow the framework established by Kim et al., with minor notational modifications to align with our specific project objective.

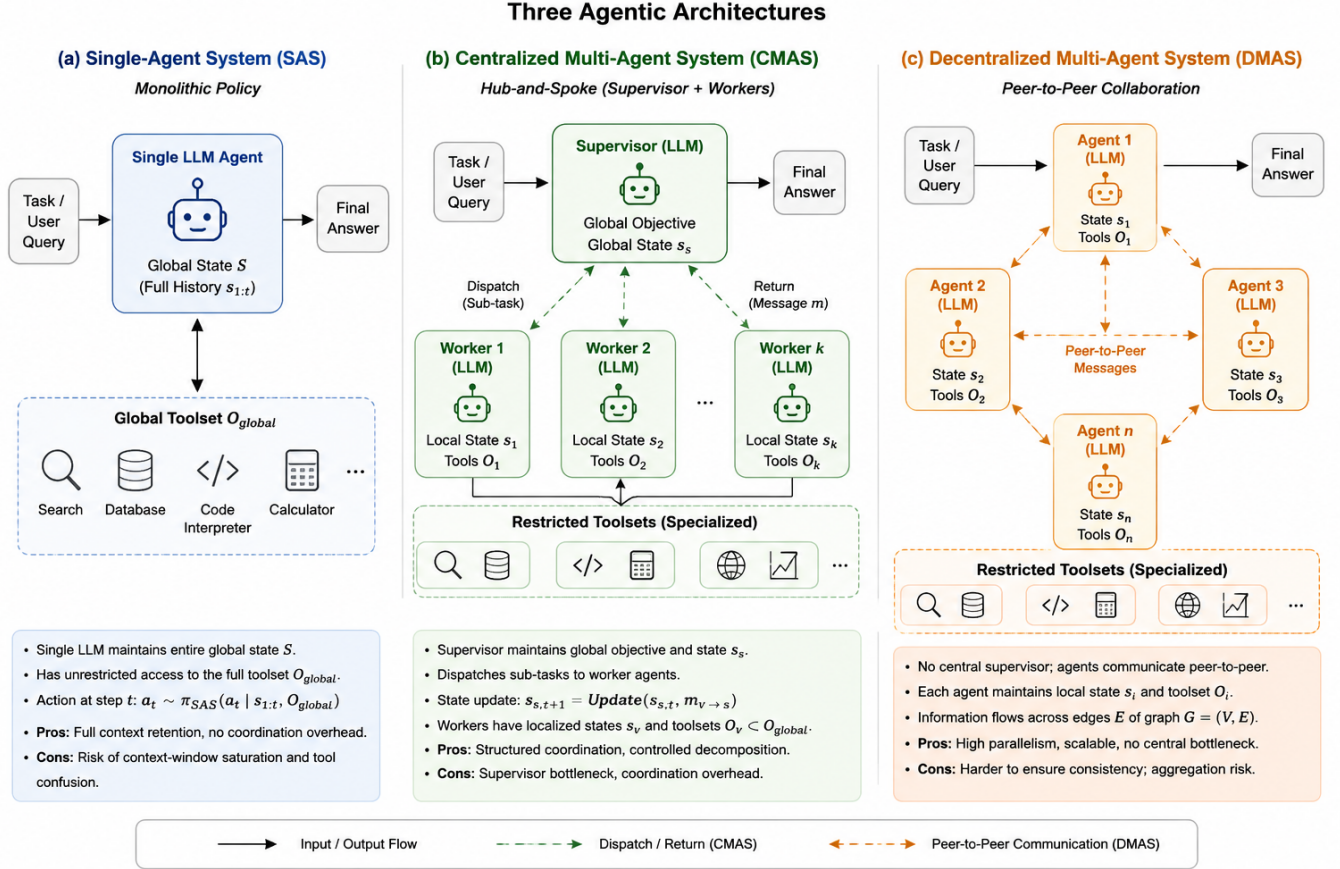


Fig. 1: Agent system overview detailing the structural distinctions between Single-Agent Systems (SAS) and Multi-Agent Systems (MAS). The architectures represent varying degrees of communication overhead, parallelization, and centralized orchestration, which directly influence the system’s Coordination Tax.

C. Formalizing Multi-Agent Systems (MAS)

A Multi-Agent System is defined as a distributed execution graph $G = (V, E)$, where each node $v \in V$ represents a specialized worker agent. In MAS, the global state space is deliberately fragmented. A worker node v possesses a localized sub-state s_v and a restricted toolset $O_v \subset O_{global}$. To solve task T , information must be passed across the edges E .

In a **Centralized MAS (CMAS)**, the graph resembles a hub-and-spoke model. A Supervisor node v_s maintains the global objective and dispatches sub-tasks to workers. The state update for the Supervisor depends on the message m returned by the worker:

$$s_{s,t+1} = \text{Update}(s_{s,t}, m_{v \rightarrow s}) \quad (7)$$

In a **Decentralized MAS (DMAS)**, parallel peers evaluate the task simultaneously, and their outputs are aggregated via a consensus or deterministic merge algorithm.

III. SOLUTION: METHODOLOGY & IMPLEMENTATION

A. Agent Topologies and Implementation

We implemented the three distinct architectural topologies. The WorkBench implementation uses LangGraph and LLM-backed ReAct agents with a configurable worker model using gpt-5.4-mini as the backbone model. For the final BrowseComp-Plus and Fin-RATE pilots, we use the same LLM-backed worker backbone over fixed local corpora so each architecture can invoke a local-search tool, but the evidence pool is fixed across SAS, CMAS, and DMAS. Since the main objective is to test out the different agent architectures across varied datasets of different nature while maintaining reproducibility, and to prevent uncontrolled token-usage and wide-variance and uncertainty from unrestricted agent environment, this preserves controlled evidence access while testing real LLM synthesis rather than raw retrieval-only approach.

SAS Implementation: We utilized a standard ReAct (Reasoning and Acting) loop for tool-use runs. The SAS was granted access to all relevant tools/domains simultaneously.

The WorkBench SAS used a recursion limit of 40, and the generic SAS graph used a recursion limit of 15 in our circuit-breaking design.

CMAS Implementation: The Centralized MAS was designed as a dynamic supervisor-worker network. In WorkBench, the supervisor evaluates the aggregated context and outputs a routing tag (e.g., `ACTION: email_agent`). The worker executes its specific ReAct loop using only tools restricted to its domain, and returns its final observation to the supervisor’s shared state. WorkBench enforced a maximum of 20 supervisor turns and 30 worker ReAct iterations; the generic CMAS graph used a 10-turn supervisor guard and 12 worker iterations.

DMAS Implementation: The Decentralized MAS splits the task across two parallel peer workers and performs a consensus merge before final evaluation. For BrowseComp-Plus and Fin-RATE, the two peers use different LLM-guided retrieval strategies over the same fixed local corpus: a full-query branch plus an alternate or shortened-query branch. A final LLM merger reconciles peer reports and produces the submitted answer.

B. The Optimization Objective

The Coordination Tax arises directly from the message passing across edges E . Every transmission requires appending previous context to a new LLM prompt, which multiplies token usage by the depth of the graph. Consequently, a static architecture assignment is inherently suboptimal in MAS.

Rather than simply maximizing accuracy at any computational cost, our formal objective is to design a dynamic routing function that maps an incoming query q to a predicted architecture $\hat{a}(q) \in \{\text{SAS}, \text{CMAS}, \text{DMAS}\}$. To establish a theoretical ground truth (the “Oracle Baseline”), we evaluate each architecture’s performance on a given query using a Composite Reward Score (R). This metric answers the important economic question: *Is the extra accuracy of a multi-agent system worth the latency and computational cost?* The goal of this router is essentially to minimize **Routing Regret**—the utility lost by not selecting the mathematically optimal Oracle $O(q)$. While exact reward weighting varies according to specific enterprise Service Level Agreements (SLAs), recent infrastructure research heavily emphasizes that test-time scaling must be strictly bounded by latency and token constraints to remain viable in production [1], [2]. To model a representative enterprise deployment where latency is the primary concern, we establish a heuristic profile of: $\alpha = 0.50$ (Accuracy), $\beta = 0.40$ (Latency), and $\gamma = 0.10$ (Token Efficiency). To ensure fair task-level comparison, latency and token costs are max normalized against the worst-performing architecture ($\max_{a \in A}$) for that specific query.

Formally, the router seeks to approximate the Oracle by maximizing the expected Composite Reward (R) over the task space:

$$\hat{a}(q) \approx O(q) = \arg \max_{a \in \{\text{SAS}, \text{CMAS}, \text{DMAS}\}} R(a, q) \quad (8)$$

where

$$\begin{aligned} R(a, q) = & \alpha \cdot \text{Acc}(a, q) \\ & - \beta \cdot \frac{\text{Lat}(a, q)}{\max_{a' \in A} \text{Lat}(a', q)} \\ & - \gamma \cdot \frac{\text{Tok}(a, q)}{\max_{a' \in A} \text{Tok}(a', q)} \end{aligned} \quad (9)$$

C. Multi-Domain Benchmarking Framework

To ensure rigorous and reproducible analysis, we adapted three industry-standard datasets into fixed, offline local schemas. This prevents architectural behavior from uncontrolled web-retrieval variance and API rate limits.

- 1) **WorkBench [15]h:** This dataset tests sequential, deterministic tool-use. It provides 26 tools across 5 simulated enterprise domains (Calendar, Email, CRM, Analytics, Project Management). We evaluated a subset of 50 tasks in an executable sandbox and compared resulting database DataFrames.
- 2) **BrowseComp-Plus [16]:** This dataset tests exploratory retrieval and source-aware context merging. We converted 30 queries and their referenced evidence documents into a fixed local corpus. We also use document identifiers with sources attached, as well as a BM25-style query retrieval serving as the deterministic fallback.
- 3) **Fin-RATE [17]:** This dataset tests advanced financial synthesis across 30 tasks. To evaluate deep reasoning, the dataset is divided into three task families: *Detail Reasoning (DR)*, *Entity/Company Comparison (EC)*, and *Longitudinal Tracking (LT)*. Such division is also employed by Jiang et al. in [17]. Grading was conducted via a GPT-4o-mini semantic judge assigning partial (0.5) or full (1.0) credit.

A major implementation challenge was adapting benchmarks from different sources into a unified SAS/CMAS/DMAS evaluation framework. WorkBench, BrowseComp-Plus, and Fin-RATE differ substantially in their original formats: WorkBench is an executable workplace tool-use environment, BrowseComp-Plus is a retrieval-heavy question-answering benchmark, and Fin-RATE evaluates financial reasoning over Securities and Exchange Commission style document chunks. To compare architectures consistently, we normalized all three into a common result schema containing task metadata, architecture output, accuracy score, latency, token usage, execution path, trace payload, and optional semantic failure labels (see Table I)

D. MAST-Inspired Failure Diagnostics

Traditional benchmarking paradigms evaluate architectures based on binary outcome success (i.e., is the final answer correct or wrong). While this is sufficient for single-agent evaluation, it is not enough to properly diagnose failures behind multi-agent systems. To properly calculate the Coordination Tax and optimize routing, the system must diagnose *why* a multi-agent system failed, rather than stating the failure.

TABLE I: Benchmark adaptation and conversion pipeline used for unified SAS/CMAS/DMAS evaluation.

Benchmark	Original Format	Converted Components	Key Preserved Metadata	Primary Evaluation Purpose
WorkBench	Interactive enterprise workflows	Task JSON + execution traces	task_id, workflow steps, tool calls	Long-horizon orchestration and routing regret evaluation
BrowseComp-Plus	Retrieval-heavy QA benchmark	Query JSONL + local corpus JSONL	query_id, gold_doc_ids, evidence_doc_ids	Source-aware retrieval and coordination analysis
Fin-RATE	Financial QA over SEC filings	QA JSONL + filing corpus JSONL	q_id, doc_id/doc_ids, company, filing year	Financial reasoning and document synthesis evaluation

TABLE II: Runtime Structural Tags and Post-Hoc Semantic Failure Labels Used in the Evaluation Pipeline

Failure Label	Layer	Implementation Type
System Design Issue: ReAct Loop Exhaustion	Runtime structural tag	Hard-coded exception check for recursion errors
System Design Issue: Context Window Overflow	Runtime structural tag	Hard-coded exception check for context errors
System Design Issue: Supervisor Turn Limit	Runtime structural tag	Hard-coded WorkBench CMAS turn-limit guard
System Design Issue: Infinite Loop	Runtime structural tag	Hard-coded CMAS supervisor turn-limit guard
System Design Issue: Loop Score	Runtime structural tag	Hard-coded repeated-dispatch score threshold
System Design Issue: Tool Explosion	Runtime structural tag	Hard-coded repeated tool-signature check
Inter-agent Misalignment: Cyclic Repetition	Runtime structural tag	Hard-coded repeated worker sequence check
Inter-agent Misalignment: Oscillating Dispatch	Runtime structural tag	Hard-coded alternating worker pattern check
Inter-agent Misalignment: Synthesis Drift	Runtime structural tag	Hard-coded redispatch-of-completed-worker guard
Worker Error / Catch-All	Runtime structural tag	Dynamic exception string rather than a stable label
Specification Failure: Requirement Miss	Post-hoc semantic label	Closed-set LLM label with heuristic fallback
Inter-agent Misalignment: Wrong Assumption	Post-hoc semantic label	LLM-selected semantic label without runtime check
Task Verification: Premature Termination	Post-hoc semantic label	Closed-set LLM label with heuristic fallback
Task Verification: Incorrect Synthesis	Post-hoc semantic label	Closed-set LLM label with heuristic fallback
Inter-agent Misalignment: Consensus Failure	Post-hoc semantic label	LLM-selected label with limited fallback
Inter-agent Misalignment: Peer Drift	Post-hoc semantic label	LLM-selected label with string-based fallback
Task Verification: Unresolved Conflict	Post-hoc semantic label	LLM-selected semantic label with fallback

Table III: The failure-analysis pipeline combines deterministic runtime structural tags with a closed-set post-hoc semantic judge inspired by the Multi-Agent System Failure Taxonomy (MAST). Runtime tags are emitted directly by orchestration guards and circuit breakers during execution, while semantic labels are assigned after failed CMAS/DMAS outputs using a constrained LLM-based classifier with lightweight heuristic fallbacks.

To achieve this, we introduce a diagnostic layer inspired by the Multi-Agent System Taxonomy (MAST) [14]. The original MAST framework categorizes MAS failures into 14 fine-grained modes across broad families—such as Specification/Design Failures, Inter-agent Misalignment, and Task Verification Failures—traditionally requiring extensive human annotation. Recognizing the constraints of automated benchmarking, we designed a project-scoped, runtime-detectable subset of MAST. Rather than attempting full MAST annotation, our subset combines runtime-detectable structural tags with lightweight post-hoc semantic labels for failed CMAS/DMAS outputs that is more suitable for our project use-case.

The diagnostic framework operates in two distinct layers:

1) *Runtime Structural Tags*: As opposed to semantic evaluation, the architecture runners continuously monitor execution state to emit structural tags via circuit breakers, loop detection

algorithms, and exception handling. These operational tags catch failures live during CMAS/DMAS execution. Examples include:

- *System Design Issue: Supervisor Turn Limit*
- *System Design Issue: Worker Context Window Overflow*
- *Inter-agent Misalignment: Cyclic Repetition / Oscillating Dispatch*

These structural tags directly capture the catastrophic token growth and exhaustive dispatch loops identified as the primary drivers of the Coordination Tax.

2) *Post-Hoc Semantic Labeling*: Because structural tags cannot easily capture semantic degradation (e.g., wrong assumptions, bad synthesis, or silent incorrect answers), we implemented a lightweight post-hoc classifier using a LLM judge. This semantic labeling runs only after benchmark grading has completed, and is only applied to failed CMAS and

DMAS outputs (e.g., when the verification score falls below a predefined threshold). SAS outputs are intentionally excluded to focus on diagnosing multi-agent coordination failures.

The semantic judge inspects the complete execution footprint, including the architecture, task, gold answer, final answer, error type, execution path, and compact trace payloads. It maps the failure to a closed-set taxonomy: *Specification Failure: Requirement Miss, Inter-agent Misalignment: Wrong Assumption Propagation, Task Verification Failure: Premature Termination, Incorrect Synthesis / Incomplete Aggregation, Consensus Failure, Peer Propagation Drift, and Unresolved Conflict at Termination*. See Table ?? for full failure labels and implementation.

To guarantee reproducibility, the judge utilizes a double backend mechanism. It first computes a heuristic fallback label from observable signals (e.g., mapping an oscillating dispatch trace to a *Consensus Failure*, or an empty final answer to a *Requirement Miss*). It then invokes an LLM with a strict prompt to output a JSON object containing the label, confidence score, rationale, and extracted evidence. If the LLM is unavailable or returns malformed JSON, the system falls back to the deterministic heuristic label. This hybrid approach ensures the Dynamic Router receives consistent diagnostic signals regarding the root cause of multi-agent failures in an automated approach.

E. The Routing Engine & Feature Engineering

We implemented two routing mechanisms to evaluate predictive dispatching. The baseline is a zero-shot rule router, which relies on explicit domain keywords and heuristic complexity triggers to assign tasks.

To test machine-learning-based dispatching for the dynamic router, we developed a Learned Meta-Agent. We extracted 14 distinct metadata features to assess task complexity before execution. These features estimate structural properties of the task without executing the full SAS/CMAS/DMAS candidates. A Logistic Regression (LR) model was trained on a 110-task pilot set. We evaluated both an unweighted model and a model with balanced class weights to account for dataset skew. Table III details the 14 features engineered to predict task architecture mapping. These features successfully characterize if the task was parallelizable as well as the communication load without requiring a full LLM invocation.

IV. NUMERICAL RESULTS AND ANALYSIS

A. Architecture Baseline Performance

Figure 2 and Table IV illustrate the baseline performance of the topologies across the benchmarking suite. The BrowseComp-Plus and Fin-RATE rows use agents with gpt-5.4-mini as the backbone model over the fixed converted corpora.

In the BrowseComp-Plus pilot, all three architectures reached the same average accuracy of 0.9333 across 30 tasks. SAS won 25 oracle decisions because it usually matched the MAS answers at lower latency and token cost. Recent literature on agentic scaling [4] suggests that decentralized systems

TABLE III: 14 Metadata Features for Learned Logistic Regression

Feature	Range/Type	Meaning
estimated_sequential_depth	1-20	Est. dependent reasoning/action stages
parallelization_factor	0.0-1.0	Task decomposability into branches
estimated_tool_count	1-50	Est. tools, APIs, or data sources needed
num_subgoals	1-50	Distinct sub-questions requested
entity_count	1-100	Independent entities/time periods
constraint_tightness	1-5	Strictness of task constraints
open_endedness	1-5	Specific vs. exploratory nature
aggregation_required	Boolean	Whether branches must be synthesized
expected_retrieval_fanout	1-100	Est. evidence batches/chunks retrieved
domain_span	1-20	Domain/tool ecosystems involved
expected_context_expansion	1-5	Expected context growth during run
final_synthesis_complexity	1-5	Difficulty of merging intermediate data
cross_branch_dependency	1-5	Inter-branch output dependency
communication_load_est.	1-5	Expected message-passing burden

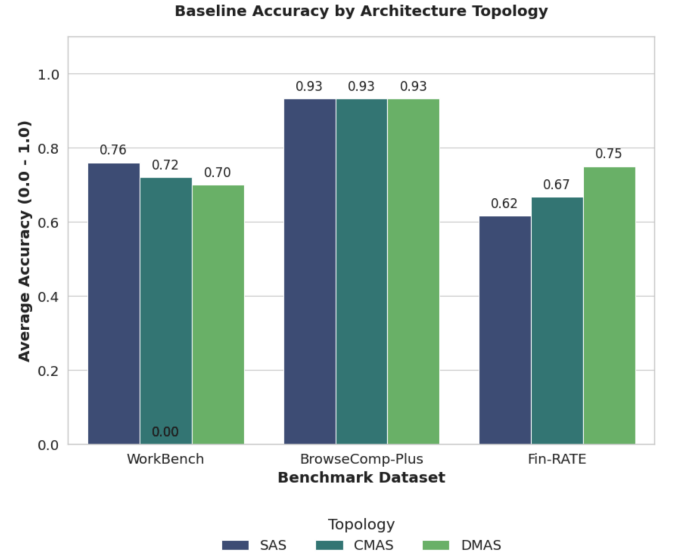


Fig. 2: Baseline accuracy across architectural topologies. The fixed-corpus LLM pilots show that SAS remains the strongest composite-oracle baseline on BrowseComp-Plus, while DMAS achieves the highest average Fin-RATE accuracy but pays additional coordination cost.

(DMAS) are typically highly suitable for open-ended, exploratory retrieval. However, because our experimental design utilizes a controlled, fixed local corpus rather than raw live-web environment, the need for expansive search parallelization was heavily reduced. Consequently, CMAS achieved the same average accuracy but won no oracle decisions, indicating that centralized supervision added pure coordination overhead without enough accuracy gain in this bounded setting. DMAS won 5 tasks, primarily when the parallel peer strategy acted as an effective cross-checker, recovering specific snippets that SAS missed while achieving comparable accuracy with lower normalized cost.

The Fin-RATE pilot changed more substantially under LLM-backed execution. DMAS achieved the highest average accuracy of around 0.7500, followed by CMAS (0.6667) and SAS (0.6167). This confirms that multi-agent decomposition can help on financial comparison and longitudinal synthesis

tasks. However, SAS still won 22 of 30 composite-oracle decisions because many tasks tied on accuracy and SAS was substantially faster and cheaper.

Notably, while scaling theories posit that Centralized Multi-Agent Systems (CMAS) excel on highly parallelizable tasks with lower tool calls—which theoretically aligns with Fin-RATE’s structure—our results yielded zero oracle wins for CMAS compared to 8 for DMAS. This unexpected finding is due to the controlled data environment. By supplying agents with a fixed context rather than requiring them to orchestrate raw retrieval tools, the hierarchical dispatching of CMAS became a strict coordination tax. DMAS, conversely, thrived because its peer-based decomposition was better suited to synthesizing and synthesizing the constrained financial details without the rigid, token-heavy overhead of a centralized supervisor.

TABLE IV: Baseline Performance & Primary Failure Modes

Benchmark	Arch.	Avg. Acc.	Oracle Wins	Primary Failure Mode / Note
WorkBench	SAS	0.7600	32 (64%)	Outcome mismatches (Unlabeled)
	CMAS	0.7200	4 (8%)	Requirement Miss
	DMAS	0.7000	14 (28%)	Incorrect Synthesis
BrowseComp-Plus	SAS	0.9333	25 (83.3%)	Imperfect scores (Unlabeled)
	CMAS	0.9333	0 (0.0%)	Requirement Miss
	DMAS	0.9333	5 (16.7%)	Requirement Miss / Incorrect Synthesis
Fin-RATE	SAS	0.6167	22 (73.3%)	Imperfect scores (Unlabeled)
	CMAS	0.6667	0 (0.0%)	Requirement Miss
	DMAS	0.7500	8 (26.7%)	Requirement Miss

The Fin-RATE dataset provides the clearest evidence that multi-agent decomposition can improve answer quality while still incurring a Coordination Tax. DMAS improved average accuracy by 0.1333 over SAS, but its additional peer calls and merge step increased latency and tokens. As a result, the composite oracle favored SAS whenever accuracy tied, while selecting DMAS for tasks where peer decomposition recovered material financial details that SAS missed.

B. Semantic Failure Analysis and Context Dilution

To understand the degradation in CMAS and DMAS, we evaluated failed multi-agent outputs utilizing our MAST-inspired semantic judge. The distribution of these failures is reflected in Figure 3.

The dominance of *Requirement Miss* reveals that distributed architectures generally fail from semantic degradation. This context dilution or fragmentation occurs when a global task is partitioned. In CMAS, worker agents optimize for localized sub-tasks and routinely discard global constraints (e.g., specific date ranges, calculations, or output formats) by the time they return data to the Supervisor. Similarly, in DMAS, parallel peers retrieve fragmented evidence that the final consensus step fails to fully verify against the original prompt. DMAS also exhibited cases of *Incorrect Synthesis*, validating that parallel peer-merging algorithms can remain lossy compared to SAS monolithic reasoning. As a result, the system produces a response that is plausible but incomplete. This vulnerability was especially exposed in the BrowseComp-Plus and Fin-RATE jobs, where intermediate workers often found relevant

evidence but failed to preserve every constraint through final synthesis.

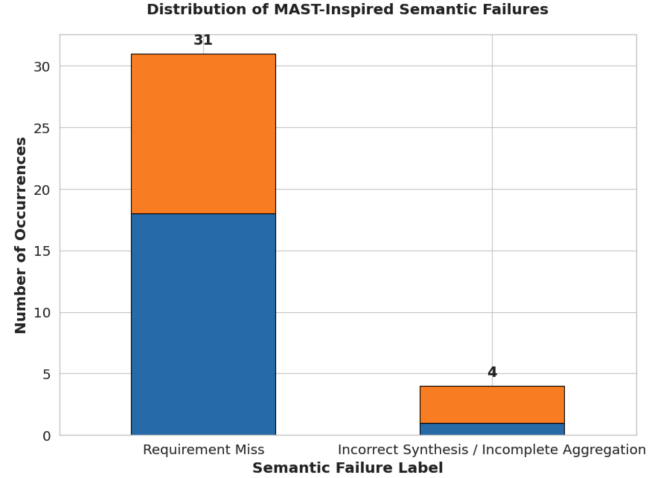


Fig. 3: Semantic failures across DMAS and CMAS architectures. The most common failure was due to requirement misses. CMAS/DMAS usually did not fail because agents disagreed or crashed, but because coordination diluted the exact requirements of the task. The system retrieved relevant-looking material, but the final answer did not satisfy the whole specification.

C. Router Evaluation

Table V presents the regret metrics for the routing policies. An important finding is the disparity between *Accuracy Regret* and the true Coordination Tax. In WorkBench, CMAS frequently generated the correct answer, which would yield an Accuracy Regret of 0.00. However, because CMAS consumed thousands of excess tokens and increased latency, the router incurred a huge hidden structural tax. Relying strictly on accuracy masks the operational inefficiency of MAS, underscoring the necessity of the Composite Oracle. For the updated BrowseComp-Plus and Fin-RATE rows, the LR policies were evaluated using the deterministic router-metadata fallback because the router metadata backend was unavailable during table generation. The learned policy comparison should therefore be interpreted as a policy-shape diagnostic rather than a freshly optimized router result.

TABLE V: Dynamic Router Evaluation (Regret Metrics)

Dataset	Router Policy	Perfect Routing	Acc. Regret	Lat. Regret(s)	Tok. Regret
WorkBench	Rule/SAS (Base)	32 / 50 (64%)	0.0800	0.4512	1232.92
	LR (Balanced)	15 / 50 (30%)	0.1400	0.8782	1300.90
	LR (Unweighted)	32 / 50 (64%)	0.0800	0.4512	1232.92
BrowseComp-Plus	Rule/SAS (Base)	25 / 30 (83.3%)	0.0333	0.2721	256.73
	LR (Balanced)	5 / 30 (16.7%)	0.0333	2.1631	28787.07
	LR (Unweighted)	25 / 30 (83.3%)	0.0333	0.2721	256.73
Fin-RATE	Rule/SAS (Base)	22 / 30 (73.3%)	0.1500	0.0000	0.00
	LR (Balanced)	8 / 30 (26.7%)	0.0333	2.9078	4048.10
	LR (Unweighted)	22 / 30 (73.3%)	0.1500	0.0000	0.00

D. Dataset Bias and Learned Router Analysis

The results reveal a serious class imbalance in the training set. Of the 110 Oracle labels in the training set, 88 favored

SAS, 14 favored DMAS, and only 8 favored CMAS. Due to this extreme skewing, the unweighted logistic regression predicts SAS-routing policy 100% of the time. Conversely, applying balanced class weights caused the router to dramatically over-amplify the minority labels and over-predict MAS classes, affecting the overall routing accuracy (Perfect Routing dropped to 30% on WorkBench).

This phenomenon is a valuable dataset finding: The tested pilot tasks adequately test tool-use syntax and single-stream retrieval, but lack sufficient parallel depth to justify the MAS Coordination Tax. Consequently, zero-shot heuristic rule routing remains more robust than lightweight machine learning models that are highly prone to sequential overfitting.

V. CONCLUSION

This research demonstrates that while multi-agent systems hold real promise for complex reasoning and parallel data synthesis, applying them indiscriminately can incur a substantial Coordination Tax. By evaluating SAS, CMAS, and DMAS topologies against a Composite Reward Oracle, we showed that single-agent state spaces still dominate the tested controlled pilot suite, but the LLM-backed Fin-RATE results also show that decentralized peers can recover accuracy on genuinely comparative financial tasks. Furthermore, our MAST-inspired telemetry exposed that distributed agents suffer from context dilution, leading to frequent requirement misses as instructions degrade across graph edges.

Ultimately, Dynamic Architecture Routing can mitigate inference regret by defaulting to single-agent topologies for sequential tasks. However, integrating learned dispatchers will require the development of massively parallelized benchmarking datasets.

A. Scope and Limitations

This evaluation should be understood as a controlled orchestration study, not as a full autonomous web-agent benchmark. The BrowseComp-Plus and Fin-RATE pilots do not simulate unrestricted web surfing. Instead, they use local converted corpora, evidence metadata, and source-aware retrieval. In the final pilots, the architectures are LLM-backed agents over those fixed corpora, so the experiment tests reasoning and coordination while still holding evidence access constant. This makes the evaluation more reproducible and better suited for routing regret analysis, but it also narrows the claim being made.

The main claim of this project is not that SAS universally dominates all possible agentic systems. Rather, the claim is that under the tested controlled evidence-access conditions, MAS must produce enough accuracy lift to overcome its coordination overhead. BrowseComp-Plus rarely provided that lift, while Fin-RATE showed a clearer accuracy benefit for DMAS on comparison and longitudinal tasks. The project evaluates how retrieval and reasoning work should be implemented across architectures, and not whether unconstrained autonomous agents can discover evidence on the open web.

It is also important to note that the primary objective of this work is to empirically validate agentic scaling laws and the resulting coordination overhead. As such, the proposed router serves as a theoretical framework for architectural selection rather than a production-grade system optimized for concurrent, high-throughput request handling.

REFERENCES

- [1] Z. Wang, T. Zhang, H. Bai, L. Hou, X. Yu, W. Liu, S. Xiang, and L. Zhu, "Faster and Better LLMs via Latency-Aware Test-Time Scaling," in *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 2025, pp. 17 124–17 137.
- [2] C. Snell *et al.*, "Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters," 2024, arXiv:2408.03314.
- [3] J. Li, Q. Zhang, Y. Yu, Q. Fu, and D. Ye, "More Agents Is All You Need," *Transactions on Machine Learning Research*, 2024.
- [4] Y. Kim, K. Gu, C. Park *et al.*, "Towards a Science of Scaling Agent Systems," 2025, arXiv:2512.08296.
- [5] X. Li, "When Single-Agent with Skills Replace Multi-Agent Systems and When They Fail," 2026, arXiv:2601.04748.
- [6] G. Park, S. Lee, and Y. Park, "Minimizing Response Latency in LLM-Based Agent Systems: A Comprehensive Survey," *IEEE Access*, vol. 14, pp. 1–20, 2026.
- [7] N. Otoum and N. Elkhaili, "Methods and Techniques of Agentic Software Engineering: A Systematic Literature Review," *IEEE Access*, vol. 14, pp. 7443–7465, 2026.
- [8] S. Chen *et al.*, "Sleipnir: Multi-agent based stock operator framework with interpretability," 2026, available on IEEE Xplore.
- [9] H. Wu, Z. He *et al.*, "ChatEDA: A Large Language Model Powered Autonomous Agent for EDA," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 10, pp. 3184–3197, 2024.
- [10] Y. Xia *et al.*, "LLM Experiments with Simulation: Large Language Model Multi-Agent System for Simulation Model Parametrization in Digital Twins," in *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2024.
- [11] A. Al-Khatib *et al.*, "LLM-Driven Multi-Agent Architectures for Intelligent Self-Organizing Networks," *IEEE Network*, 2025.
- [12] J. Liao *et al.*, "AutoForma: A Large Language Model-Based Multi-Agent for Computer-Automated Design," in *2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2024.
- [13] L. Wu *et al.*, "A Large Language Model-Driven Multi-Agent Human-Machine Interaction Framework," in *2025 10th International Conference on Intelligent Informatics and Biomedical Sciences (ICIIBMS)*, 2025.
- [14] Z. Hasan, A. Payani *et al.*, "Why Do Multi-Agent LLM Systems Fail?" 2025, arXiv:2503.13657, <https://sites.google.com/berkeley.edu/mast/>.
- [15] O. Styles *et al.*, "WorkBench: a Benchmark Dataset for Agents in a Realistic Workplace Setting," 2024, arXiv:2405.00823.
- [16] Z. Chen, X. Ma, S. Zhuang, P. Nie, K. Zou, A. Liu, J. Green, K. Patel, R. Meng, M. Su, S. Sharifmoghaddam, Y. Li, H. Hong, X. Shi, X. Liu, N. Thakur, C. Zhang, L. Gao, W. Chen, and J. Lin, "Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent," 2025. [Online]. Available: <https://arxiv.org/abs/2508.06600>
- [17] Y. Jiang, J. Chen, E. Makri, J. Chen, P. Li, A. Maatouk, L. Tassioulas, E. Brenner, B. Xiang, and R. Ying, "Fin-rate: A real-world financial analytics and tracking evaluation benchmark for llms on sec filings," 2026. [Online]. Available: <https://arxiv.org/abs/2602.07294>